

1 Detection hai giai đoạn nuốt dần heuristic (Two-Stage Detection)

PHỤ THUỘC C/13-kien-truc-backbone/01-dong-cnn (feature map, stride, trường tiếp nhận (*receptive field*)); B/09-thiet-ke-ham-mat-mat (loss đa nhiệm); B/05-thiet-ke-bai-toan (bộ ba đầu vào – đầu ra – tín hiệu giám sát).

TÓM TẮT

Mục này theo dấu bốn thể hệ detection hai giai đoạn: R-CNN, Fast R-CNN, Faster R-CNN với FPN, và Mask R-CNN. Chúng không phải bốn ý tưởng rời rạc, mà là bốn lần thay một bước thủ công bằng một khối học được. Đọc xong, cầm bất kỳ paper detection nào bạn cũng chỉ ra được hai thứ: khối nào nằm trong tầm với của gradient, và khối nào là heuristic cố định. Từ đó bạn đoán được bước cải tiến hợp lý tiếp theo. Đó là kỹ năng dự đoán, không phải ghi nhớ. Nếu chỉ nhớ một điều: ranh giới “cái gì nhận được gradient” dịch xuống qua từng thể hệ. Hai thứ sống sót được là triết phi cực đại và cách gán mẫu. Chúng sống sót vì lý do cấu trúc, không phải ngẫu nhiên, nên gỡ chúng đòi hỏi đổi phát biểu bài toán chứ không đổi kiến trúc.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, ánh xạ ảnh đầu vào thành dự đoán; “huấn luyện” là tìm bộ tham số làm dự đoán khớp nhãn nhất có thể.
- **gradient** — vector đạo hàm của hàm mất mát theo từng tham số; nó chỉ hướng phải chỉnh tham số để lỗi giảm. Một khối “không nhận được gradient” là khối không tự chỉnh được, dù nó gây ra lỗi.
- **backbone** — phần mạng dùng chung để biến ảnh thành *feature map* — một lưới vector đặc trưng, mỗi ô mô tả nội dung của một vùng ảnh. Phần gắn phía trên để trả lời một bài toán cụ thể gọi là *head*.
- **stride** — tỉ lệ thu nhỏ giữa ảnh và feature map. Stride 16 nghĩa là một ô đặc trưng ứng với một vùng 16×16 pixel ảnh, nên mọi tọa độ tính trên feature map đều thô tới 16 pixel.
- **IoU** — diện tích giao chia diện tích hợp của hai khung chữ nhật; bằng 1 khi trùng khít, 0 khi rời nhau. Đây là thước đo “khung dự đoán có đúng chỗ không”.
- **anchor** — một khung chữ nhật mẫu, kích thước và tỉ số cạnh định sẵn, rải đều trên mọi vị trí của feature map. Mô hình không đoán khung từ số không mà đoán *độ lệch* so với anchor gần nhất — một bài toán dễ hơn nhiều.
- **proposal (đề xuất vùng)** — một vùng ảnh ứng viên “có thể chứa vật gì đó”, sinh ra ở giai đoạn một để giai đoạn hai chỉ phải phân loại vài trăm vùng thay vì hàng chục nghìn.
- **NMS** — bước hậu xử lý xoá các khung trùng lặp: giữ khung điểm cao nhất, xoá mọi khung có IoU với nó vượt ngưỡng, lặp lại. Nó là thuật toán viết tay, không học được.
- **mAP** — thước đo chuẩn của detection: với mỗi lớp, vẽ đường precision-recall rồi lấy diện tích dưới đường đó, sau đó trung bình trên mọi lớp và trên nhiều ngưỡng IoU.

Đặt vấn đề

Phân loại ảnh có một đặc tính dễ chịu mà ta ít khi để ý: đầu ra của nó có kích thước cố định. Một ảnh vào, một vector K chiều ra, và toàn bộ bộ máy huấn luyện bằng *gradient* chạy trơn tru vì **hàm mất mát (loss function)** được định nghĩa trên một đối tượng có hình dạng biết trước. **Phát hiện vật thể (object detection)** phá vỡ đúng đặc tính đó. Số vật thể trong ảnh thay đổi từ không tới hàng trăm, mỗi vật cần một nhãn và một vị trí, nên đầu ra là một **tập hợp (set)** có lực lượng thay đổi. Không có cách hiển nhiên nào viết một hàm mất mát khả vi trên tập hợp, và đây chính là nút thắt mà toàn bộ lịch sử detection xoay quanh.

Cách né đầu tiên, và trong nhiều năm là cách duy nhất, là biến bài toán tập hợp thành nhiều bài toán phân loại: đề xuất trước một tập ứng viên vùng ảnh, rồi phân loại từng ứng viên độc lập.

Mẹo này giải quyết được vấn đề hình dạng đầu ra, nhưng nó đẩy độ khó sang chỗ khác — sang câu hỏi *ai sinh ra các ứng viên đó*. Trong R-CNN, câu trả lời là một thuật toán phân đoạn thủ công tên là *selective search*, thứ không có tham số nào học được và không bao giờ nhận được một chút *gradient* nào từ lỗi cuối cùng.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper detection bất kỳ và chỉ ra ngay được ba thứ: khối nào nằm trong tầm với của gradient, khối nào là heuristic cố định, và vì thế bước cải tiến hợp lý tiếp theo là gì. Đó là một kỹ năng dự đoán, không phải kỹ năng ghi nhớ.

TRỰC GIÁC

Hãy hình dung hệ thống như một xưởng máy mà một số máy được nối vào bảng điều khiển trung tâm, còn một số thì quay tay. Máy nối vào bảng điều khiển tự chỉnh lại khi sản phẩm cuối bị lỗi; máy quay tay thì không, dù nó có gây ra lỗi hay không. “Nuốt một heuristic vào mạng” theo nghĩa cơ học chính xác là *nối thêm một máy vào bảng điều khiển*: biến một bước không khả vi, không tham số, thành một khối khả vi nhận được gradient từ hàm mất mát detection. Vì thế mỗi lần nuốt, hệ thống không chỉ nhanh hơn (bớt một công đoạn rời) mà còn chính xác hơn (thêm một khối được tối ưu cho đúng mục tiêu cuối).

A. Bốn bước nuốt

I Mạch 1 — detection hai giai đoạn

- 1 Vấn đề** không thể chấm điểm một tập hợp có lực lượng thay đổi.

Giải pháp R-CNN cắt khoảng hai nghìn vùng ứng viên do *selective search* sinh ra. Mỗi vùng được warp về kích thước cố định rồi cho qua CNN để lấy đặc trưng. Một SVM phân loại, và một bộ hồi quy riêng tinh chỉnh khung [1].

Giả định các vùng đáng quan tâm đều nằm trong tập ứng viên; nếu selective search bỏ sót thì không gì cứu được.

Giá phải trả mỗi ảnh phải chạy CNN hai nghìn lần, huấn luyện chia làm ba pha rời rạc, và không pha nào tối ưu cho mục tiêu của pha kia.
- 2 Vấn đề** hai nghìn lượt chạy CNN cho cùng một ảnh là lãng phí gần như toàn bộ.

Giải pháp Fast R-CNN đảo thứ tự: chạy *backbone* một lần trên cả ảnh, rồi với mỗi vùng ứng viên chỉ cắt phần tương ứng trên feature map bằng *RoIPooling* và cho qua vài tầng **kết nối đầy đủ** (*fully connected*) [2]. Phân loại và hồi quy khung được gộp thành một **hàm mất mát đa nhiệm** (*multi-task loss*) duy nhất, huấn luyện một pha.

↔ **ràng buộc bởi** B/09 — trọng số giữa hạng thức phân loại và hạng thức hồi quy là một siêu tham số ẩn, và chương thiết kế hàm mất mát giải thích vì sao nó không thể chọn tùy tiện.

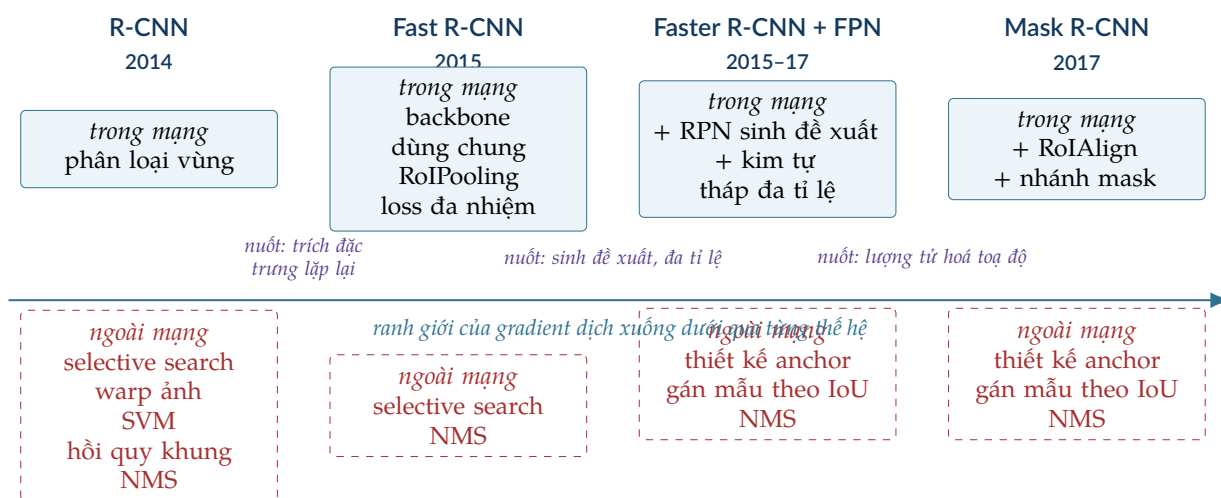
Giả định đặc trưng dùng chung cho mọi vùng vẫn đủ tốt cho từng vùng — đúng khi các vùng chồng lấn nhiều, và chúng chồng lấn thật.

Vấn đề mới khi phần tính toán nặng đã bị loại bỏ, selective search trở thành nút cổ chai.
- 3 Vấn đề** bộ sinh đề xuất là công đoạn quay tay cuối cùng còn nặng.

Giải pháp Faster R-CNN thay nó bằng **mạng đề xuất vùng** (*region proposal network, RPN*), một mạng con trượt trên chính feature map đã có, tại mỗi vị trí dự đoán độ khách quan và độ lệch khung so với một tập *anchor* định sẵn [3]. Vì RPN dùng chung backbone, chi phí sinh đề xuất gần như bằng không.

Giả định tập anchor phủ đủ dày dài tỉ lệ và tỉ số cạnh của vật thể trong dữ liệu.

- Giá phải trả** anchor trở thành một **siêu tham số** (*hyperparameter*) nhạy và *mang theo thống kê của tập dữ liệu gốc*; đổi miền ảnh thì phải thiết kế lại.
- 4 Vấn đề** một feature map duy nhất không phục vụ nổi mọi tỉ lệ. Feature ở tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thô, feature nông thì ngược lại; vật nhỏ vì thế bị thiệt kép.
- Giải pháp** **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) dựng một kim tự tháp bằng nhánh *top-down* cộng với kết nối ngang, để mọi tầng đều vừa sâu về ngữ nghĩa vừa đúng độ phân giải [4].
- Hệ quả** đây là lần đầu “đa tỉ lệ” được đưa vào kiến trúc thay vì đưa vào pipeline dưới dạng chạy ảnh ở nhiều kích thước.
- ↔ **tiền đề** C/13/01 — stride và trường tiếp nhận của backbone chính là thứ tạo ra mâu thuẫn ngữ nghĩa–độ phân giải mà FPN sinh ra để gỡ.
- 5 Vấn đề** RoIPooling làm tròn toạ độ, và việc làm tròn đó phá hỏng mọi bài toán cần độ chính xác dưới pixel.
- Giải pháp** Mask R-CNN thay bằng **RoIAlign**: không làm tròn, lấy mẫu song tuyến tại các điểm có toạ độ thực, rồi thêm một nhánh dự đoán **mặt nạ** (*mask*) nhị phân song song với nhánh phân loại [5].
- Hệ quả** nhánh mask gần như miễn phí về ý tưởng nhưng chỉ hoạt động tốt *sau khi* lỗi lượng tử hoá được gỡ bỏ — một minh hoạ sắc nét rằng chi tiết cài đặt có thể chặn đứng cả một bài toán mới.

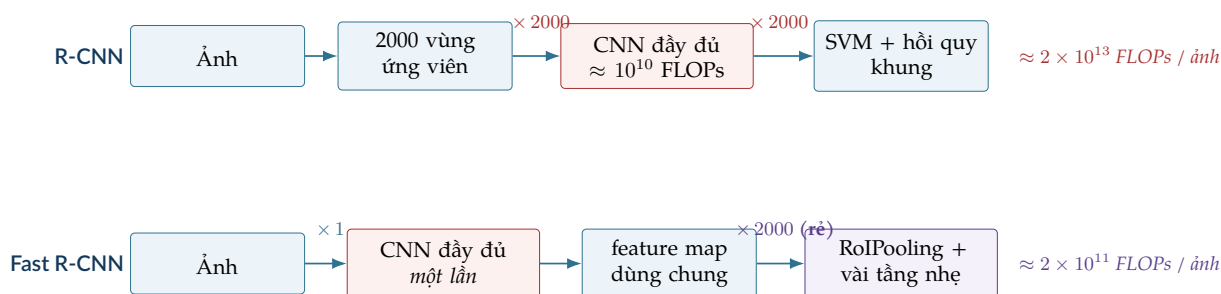


Hình 1. Dòng thời gian của họ hai giai đoạn, vẽ theo trục “cái gì nhận được gradient”. Khối trên đường kẻ nằm trong tầm với của hàm mất mát detection và tự chỉnh; khối dưới đường kẻ là heuristic cố định. Nhấn tím giữa hai cột ghi rõ mỗi bước nuốt được heuristic nào. Qua bốn thế hệ, phần dưới teo dần nhưng NMS và cách gán mẫu thì ở lại — đó là dư địa mà Mục 2 khai thác.

Hình 1 cho thấy điều mà một bảng thời gian không cho thấy: cái còn sót lại sau bốn bước không phải là ngẫu nhiên. **Triệt phi cực đại** (*non-maximum suppression, NMS*) sống sót vì nó thao tác trên *quan hệ giữa các dự đoán* chứ không trên một dự đoán đơn lẻ, mà loss của kiến trúc này lại chấm điểm từng dự đoán độc lập. Cách **gán mẫu** (*label assignment*) sống sót vì nó là một phần của *định nghĩa* loss chứ không phải một khối tính toán — không có gì để “nuốt” theo nghĩa thông thường. Hai chỗ đó đòi hỏi đổi phát biểu bài toán, và đó đúng là việc DETR làm ở mục sau.

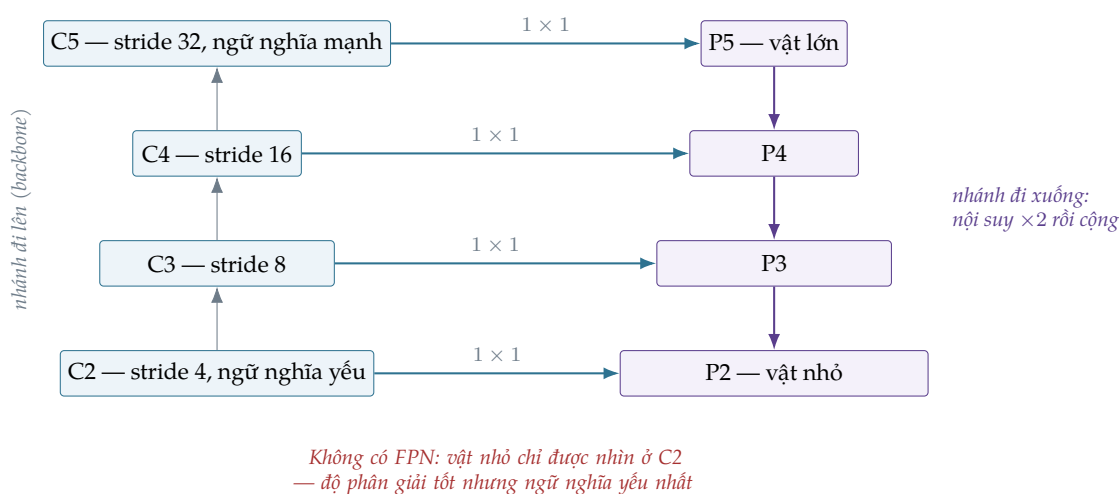
B. Ba phép tính nhỏ giải thích ba bước nhảy

- Vì sao chia sẻ feature map lại nhanh hơn hai bậc. Giả sử backbone tốn khoảng 10^{10} FLOPs cho một ảnh 224×224 .
 - R-CNN chạy nó cho từng vùng trong $N = 2000$ vùng: 2×10^{13} FLOPs mỗi ảnh.
 - Fast R-CNN chạy backbone một lần trên ảnh lớn hơn (giả sử gấp mười, 10^{11}), rồi mỗi vùng chỉ qua vài tầng nhẹ cỡ 10^8 , tổng 2×10^{11} .
 - Tỉ số khoảng $2 \times 10^{13} / 3 \times 10^{11} \approx 70$ lần, và thực tế còn lớn hơn vì R-CNN phải ghi/đọc đặc trưng ra đĩa. *Ý nghĩa*: chi phí chuyển từ $O(N)$ lượt backbone sang $O(1)$ lượt backbone cộng $O(N)$ lượt đầu nhẹ — đó mới là điều cần nhớ, không phải con số.



Hình 2. Cùng số vùng ứng viên, cùng backbone, khác nhau ở thứ tự phép cắt và phép trích đặc trưng. R-CNN đặt phép cắt trước nên khối đất nhất bị nhân với N ; Fast R-CNN đặt phép cắt sau nên khối đất chỉ chạy một lần và chỉ phần nhẹ mới bị nhân. Toàn bộ khoảng cách hai bậc độ lớn nằm ở chỗ đổi chỗ hai mũi tên, không ở chỗ nào khác.

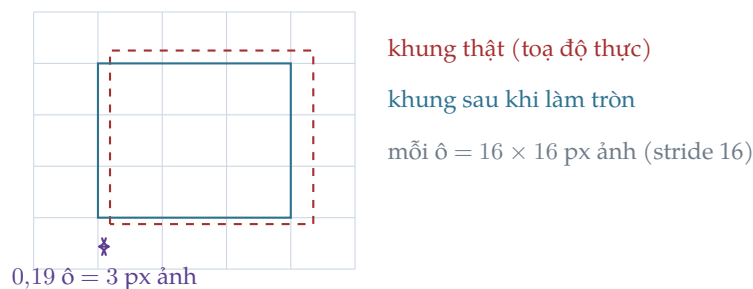
Hình 2 là dạng nhìn thấy được của phép tính vừa rồi, và nó cũng là mẫu hình đáng nhớ: khi một pipeline lập một khối đất, hãy hỏi liệu có đẩy được khối đó ra ngoài vòng lặp hay không. **Vì sao làm tròn lại đầu đến thế.** Backbone thông dụng có *stride* 16, nên một pixel feature ứng với ô 16×16 pixel ảnh. Khung có cạnh trái tại $x = 19$ rơi vào tọa độ feature $19/16 = 1,1875$; RoIPooling làm tròn xuống 1, tức dịch biên $0,1875 \times 16 = 3$ pixel ảnh gốc, rồi chia ô pooling làm tròn lần thứ hai. Ba pixel là nhiều với người cao 200 pixel, nhưng là 10% kích thước với một vật 30 pixel. Đó là lý do RoIAlign gần như không đổi gì ở ngưỡng **giao trên hợp** (*intersection over union, IoU*) lỏng nhưng cải thiện mạnh ở ngưỡng chặt và với nhánh mask [5].



Hình 3. FPN gộp hai thứ vốn nằm ở hai đầu đối lập của backbone. Đi lên, độ phân giải giảm còn ngữ nghĩa mạnh dần; đi xuống, ngữ nghĩa của tầng sâu được nội suy trả về từng mức và cộng vào đặc trưng nông qua kết nối ngang 1×1 . Kết quả là mọi mức P_i đều vừa đủ sâu vừa đủ mịn, nên vật nhỏ không còn bị thiệt kép.

- Hình 3 làm rõ vì sao đây là một bước “nuốt heuristic”: trước FPN, cách xử lý đa tỉ lệ là chạy *cả ảnh* ở nhiều kích thước rồi gộp kết quả — một vòng lặp bên ngoài mạng, tốn gấp nhiều lần. FPN đưa đúng chức năng đó vào trong một nhánh có tham số học được.

FPN gán vùng vào tầng nào. Công thức trong bài gốc là $k = \lfloor k_0 + \log_2(\sqrt{wh}/224) \rfloor$ với $k_0 = 4$. Nó nói gì: vật càng nhỏ thì càng được đẩy về tầng có độ phân giải cao. Khung 112×112 cho $\log_2(112/224) = -1$ nên $k = 3$; khung 448×448 cho $k = 5$. Đáng chú ý là công thức này vẫn là heuristic thủ công nằm ngoài gradient — lại một vòng nữa của cùng mẫu hình.



Hình 4. Vì sao RoIPooling làm hỏng bài toán mask. Trên lưới đặc trưng stride 16, việc làm tròn toạ độ khung về ô nguyên dịch biên đi tới vài pixel trên ảnh gốc; sai số này cố định, không giảm khi huấn luyện lâu hơn, và tỉ lệ nghịch với kích thước vật. RoIAlign lấy mẫu song tuyến tại toạ độ thực nên không sinh ra khoảng lệch màu tím.

Hình 4 là một lời nhắc quen thuộc với người làm hình học. Đây đúng là lỗi lượng tử hoá mà mọi hệ bám đặc trưng dưới pixel đều phải xử lý. Cách chữa cũng giống hệt: nội suy thay vì làm tròn, đúng như một bộ *tracker* KLT tinh chỉnh vị trí đặc trưng.

C. Giới hạn và trường hợp hỏng

Họ hai giai đoạn hỏng theo ba kiểu rất đặc trưng, và mỗi kiểu có một triệu chứng quan sát được:

- Cảnh đông đúc.** Khi hai vật cùng lớp chồng lấn nhiều hơn ngưỡng NMS, một trong hai chắc chắn bị xoá. *Triệu chứng:* hạ ngưỡng thì mất vật, nâng ngưỡng thì sinh trùng lặp — không có giá trị nào cứu được cả hai, vì vấn đề nằm ở phát biểu chứ không ở tham số.
- Miền anchor lệch.** Tập anchor mã hoá phân bố tỉ lệ và tỉ số cạnh của dữ liệu huấn luyện. Đưa sang ống kính góc rộng, ảnh *fisheye*, hay lớp vật dài mảnh thì recall của RPN sụt trước khi bộ phân loại kịp nói gì. *Triệu chứng:* recall giai đoạn đề xuất thấp trong khi độ chính xác trên các đề xuất còn lại vẫn cao.
- Độ trễ không ổn định.** Số đề xuất qua ngưỡng thay đổi theo ảnh, nên thời gian giai đoạn hai dao động theo nội dung cảnh. *Triệu chứng:* phân vị 99 của độ trễ cao hơn nhiều so với trung vị — một tính chất rất khó chịu khi bạn phải cam kết ngân sách mili-giây cho mỗi khung hình, đúng kiểu rằng buộc cứng đã bàn ở B/03-giai-phau-he-thong.

CẠM BẤY

Rất nhiều so sánh “kiến trúc A tốt hơn B 2 mAP” thực chất là so sánh hai công thức huấn luyện: số epoch, **tăng cường dữ liệu** (*data augmentation*), tỉ lệ mẫu dương/âm trong mỗi *batch*, ngưỡng IoU khi gán mẫu. Trước khi tin một cải tiến kiến trúc, hãy tìm xem phần gán mẫu và recipe có được giữ nguyên không — và trong đa số paper, thông tin đó nằm trong code chứ không nằm trong bài.

↔ **hệ quả của** E/24 — muốn tách phần đóng góp của một khối khỏi phần đóng góp của recipe thì phải ablate đúng một thứ — đó là phương pháp, không phải trực giác.

Công cụ như [dbolya/tide](#) tách mAP thành các loại lỗi thành phần [6], [7], và thường phát hiện rằng hai mô hình “chênh 2 mAP” hỏng theo hai kiểu hoàn toàn khác nhau.

CẦN NHỚ

Mỗi bước tiến của detection hai giai đoạn là một heuristic bị kéo vào trong tầm với của gradient. Muốn đoán bước tiếp theo của một dòng mô hình bất kỳ, kể cả *SLAM*, hãy liệt kê những gì còn nằm *ngoài* vòng tối ưu. Rồi hỏi cái nào trong số đó viết lại được thành một khối khả vi. [T3]

D. Kết nối

Mục tiếp theo (C/14-bai-toan-cv-loi/02-detection-mot-giai-doan) là mặt đối lập: bỏ hẳn giai đoạn đề xuất để đổi lấy tốc độ, và trả giá bằng một vấn đề mất cân bằng mà mục này chưa gặp. C/13-kien-truc-backbone/01-dong-cnn là tiền đề: RoIPooling, stride và trường tiếp nhận chỉ hiểu được sau khi nắm cơ học của *convolution*. B/09-thiet-ke-ham-mat-mat giải thích vì sao gộp phân loại và hồi quy vào một loss đa nhiệm không hiển nhiên đúng. E/24-thi-nghiem-va-ablation là nơi kiểm chứng cạm bẫy vừa nêu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Fast R-CNN nhanh hơn R-CNN nhiều bậc trong khi cả hai dùng cùng backbone và cùng số vùng ứng viên?
2. RoIAlign khác RoIPooling ở đúng một chỗ. Vì sao chỗ đó ảnh hưởng tới nhánh mask mạnh hơn nhiều so với nhánh phân loại?
3. Miền ảnh cụ thể nào làm tập anchor mặc định của COCO sụt recall, và triệu chứng quan sát được ở đó là gì?
4. Vì sao NMS và cách gán mẫu là hai thứ duy nhất sống sót qua cả bốn thế hệ? Hai lý do đó khác nhau ở điểm nào?
5. Nếu bạn phải cam kết một ngân sách mili-giây cố định cho mỗi khung hình, kiến trúc hai giai đoạn gây khó khăn gì mà một giai đoạn không gây?

Nguồn

- [1] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.
- [2] R. Girshick, “Fast R-CNN,” in *ICCV*, 2015.
- [3] S. Ren, K. He, R. Girshick, et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *NeurIPS*, 2015.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [5] K. He, G. Gkioxari, P. Dollár, et al., “Mask R-CNN,” in *ICCV*, 2017.
- [6] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [7] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.